Introduction to Cryptography

Lecture 16: RC4 Algorithm

Dr. Siddhant Thapliyal

Department of Computer Science and Engineering
Graphic Era (Deemed to be University), Dehradun

What is a Stream Cipher?

A stream cipher encrypts data one unit at a time.
Usually, the unit is:

1 bit or 1 byte

A stream cipher generates a key stream:

$$k_1, k_2, k_3, \ldots$$

The key stream is combined with plaintext using XOR:

$$C_i = P_i \oplus k_i$$

Decryption is:

$$P_i = C_i \oplus k_i$$

Block Cipher vs Stream Cipher

Feature	**Block Cipher**	**Stream Cipher**
Processing	Fixed-size blocks	Continuous stream
Unit	Block of bits	Bit or byte
Example	DES, AES	RC4
Operation	Block transformation	Key stream XOR
Speed	Often fast	Usually very fast

What is RC4?

RC4 is a stream cipher designed by Ron Rivest in 1987.

- RC4 stands for **Rivest Cipher 4**.
- It is a byte-oriented stream cipher.
- It uses a variable-length key.
- It generates a pseudo-random key stream.
- Encryption and decryption are performed using XOR.

$$\boxed{\text{Plaintext Byte} \oplus \text{RC4 Key Stream Byte} = \text{Ciphertext Byte}}$$

Historical Use of RC4

RC4 was historically used in:

- SSL/TLS
- WEP
- WPA
- Some legacy software systems

Modern Note

RC4 is now considered insecure for modern cryptographic use and should not be used in new systems.

Why Study RC4?

Even though RC4 is no longer recommended, it is important because:

- It is simple to understand.
- It demonstrates stream cipher design.
- It shows key scheduling and key-stream generation.
- It was widely deployed historically.
- It teaches why algorithm design and protocol use both matter.

Basic RC4 Idea

RC4 maintains a state vector:

$$S[0], S[1], S[2], \ldots, S[255]$$

At all times, S contains a permutation of:

$$0, 1, 2, \ldots, 255$$

RC4 repeatedly permutes this state and produces one key-stream byte at a time.

$$\boxed{S \rightarrow \text{Key Stream Byte} \rightarrow \text{XOR with Data}}$$

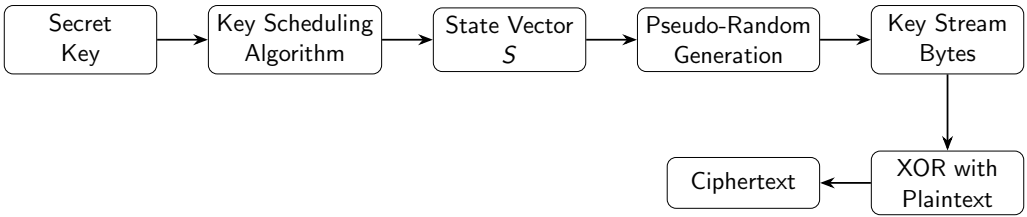
Main Components of RC4

RC4 consists of two main algorithms:

1. **Key Scheduling Algorithm**
   - Initializes and permutes the state vector S.
2. **Pseudo-Random Generation Algorithm**
   - Generates the key stream.

$$\boxed{\text{Secret Key} \rightarrow \text{KSA} \rightarrow S \rightarrow \text{PRGA} \rightarrow \text{Key Stream}}$$

RC4 High-Level Flow



RC4 State Vector S

The state vector S has 256 entries.

$$S[0], S[1], \ldots, S[255]$$

Initially:

$$S[i] = i$$

for:

$$0 \leq i \leq 255$$

Therefore:

$$S = [0, 1, 2, 3, \ldots, 255]$$

Important

The state vector is repeatedly permuted during RC4 operation.

RC4 Key

RC4 uses a variable-length key.
The key can range from:

1 byte

to:

256 bytes

That is:

8 bits

to:

2048 bits

If the key length is:

Temporary Vector T

RC4 creates a temporary vector T of size 256.

$$T[0], T[1], \ldots, T[255]$$

The key is repeated to fill T.

$$\boxed{T[i] = K[i \bmod \text{keylen}]}$$

This means the key bytes are reused cyclically until all 256 positions of T are filled.

RC4 Initialization

Initialization begins with:

$$S[i] = i$$

and:

$$T[i] = K[i \bmod \text{keylen}]$$

for:

$$i = 0, 1, 2, \ldots, 255$$

$$\boxed{S = [0, 1, 2, \ldots, 255]}$$

$$\boxed{T = \text{repeated key bytes}}$$

Key Scheduling Algorithm

The Key Scheduling Algorithm permutes S using T.

$$j = 0$$

For:

$$i = 0 \text{ to } 255$$

perform:

$$j = (j + S[i] + T[i]) \bmod 256$$

Then swap:

$$S[i] \leftrightarrow S[j]$$

After this process, S becomes a key-dependent permutation.

KSA Pseudocode

for $i = 0$ **to** 255

$$S[i] = i$$

$$T[i] = K[i \bmod \text{keylen}]$$

$$j = 0$$

for $i = 0$ **to** 255

$$j = (j + S[i] + T[i]) \bmod 256$$

$$\text{Swap}(S[i], S[j])$$

Output: Initial key-dependent permutation of S

Why Does RC4 Use Swapping?

RC4 changes the state vector using only swaps.

$$\text{Swap}(S[i], S[j])$$

A swap preserves all values in S.
Therefore, S remains a permutation of:

$$0, 1, 2, \ldots, 255$$

Important

RC4 never removes or duplicates values in S; it only rearranges them.

Toy Example: Smaller RC4 State

Real RC4 uses 256 entries.
For classroom understanding, let us use a toy state of 8 entries:

$$S = [0, 1, 2, 3, 4, 5, 6, 7]$$

Assume:

$$K = [3, 1, 4]$$

Then:

$$\text{keylen} = 3$$

and:

$$T[i] = K[i \bmod 3]$$

Toy Example: Construct T

Given:

$$K = [3, 1, 4]$$

and state size 8:

$$T[i] = K[i \bmod 3]$$

i	0	1	2	3	4	5	6	7
$T[i]$	3	1	4	3	1	4	3	1

Initial state:

$$S = [0, 1, 2, 3, 4, 5, 6, 7]$$

Toy KSA: First Iteration

Initial:

$$S = [0, 1, 2, 3, 4, 5, 6, 7]$$

$$j = 0$$

For $i = 0$:

$$j = (j + S[0] + T[0]) \bmod 8$$

$$j = (0 + 0 + 3) \bmod 8 = 3$$

Swap:

$$S[0] \leftrightarrow S[3]$$

New state:

Toy KSA: Second Iteration

Current:

$$S = [3, 1, 2, 0, 4, 5, 6, 7]$$

$$j = 3$$

For $i = 1$:

$$j = (3 + S[1] + T[1]) \bmod 8$$

$$j = (3 + 1 + 1) \bmod 8 = 5$$

Swap:

$$S[1] \leftrightarrow S[5]$$

New state:

Toy KSA: Third Iteration

Current:

$$S = [3, 5, 2, 0, 4, 1, 6, 7]$$

$$j = 5$$

For $i = 2$:

$$j = (5 + S[2] + T[2]) \bmod 8$$

$$j = (5 + 2 + 4) \bmod 8 = 3$$

Swap:

$$S[2] \leftrightarrow S[3]$$

New state:

After Key Scheduling

After the KSA:

$$S$$

is no longer:

$$[0, 1, 2, \ldots, 255]$$

It becomes a key-dependent permutation.

This means different keys produce different initial states.

$$\boxed{\text{Key} \rightarrow \text{Different permutation of } S}$$

Pseudo-Random Generation Algorithm

After KSA, RC4 generates the key stream.
Initialize:

$$i = 0, \qquad j = 0$$

Repeatedly perform:

$$i = (i + 1) \bmod 256$$

$$j = (j + S[i]) \bmod 256$$

Swap:

$$S[i] \leftrightarrow S[j]$$

Then:

$$t = (S[i] + S[j]) \bmod 256$$

PRGA Pseudocode

$$i = 0, \quad j = 0$$

while true

$$i = (i + 1) \bmod 256$$

$$j = (j + S[i]) \bmod 256$$

$$\text{Swap}(S[i], S[j])$$

$$t = (S[i] + S[j]) \bmod 256$$

$$k = S[t]$$

$$\boxed{k = \text{next key-stream byte}}$$

RC4 Key Stream Generation

The PRGA produces:

$$k_1, k_2, k_3, \ldots$$

These values form the RC4 key stream.
Each key-stream byte is XORed with one plaintext byte.

$$C_i = P_i \oplus k_i$$

Decryption uses:

$$P_i = C_i \oplus k_i$$

because:

$$(P_i \oplus k_i) \oplus k_i = P_i$$

Toy PRGA Example

Use toy state size 8.
Suppose after KSA:

$$S = [3, 5, 0, 2, 4, 1, 6, 7]$$

Initialize:

$$i = 0, \qquad j = 0$$

First key-stream byte:

$$i = (0 + 1) \bmod 8 = 1$$

$$j = (0 + S[1]) \bmod 8 = 5$$

Swap:

$$S[1] \leftrightarrow S[5]$$

Toy PRGA Example Continued

After swap:

$$S = [3, 1, 0, 2, 4, 5, 6, 7]$$

Now:

$$t = (S[i] + S[j]) \bmod 8$$

Here:

$$i = 1, \quad j = 5$$

$$t = (S[1] + S[5]) \bmod 8$$

$$t = (1 + 5) \bmod 8 = 6$$

Therefore:

RC4 Encryption

Encryption is very simple.
For each plaintext byte:

$$P_i$$

RC4 generates a key-stream byte:

$$k_i$$

Then:

$$\boxed{C_i = P_i \oplus k_i}$$

$$\text{Plaintext Byte} \oplus \text{Key Stream Byte} = \text{Ciphertext Byte}$$

RC4 Decryption

Decryption uses the same XOR operation.
Given ciphertext byte:

$$C_i$$

and the same key-stream byte:

$$k_i$$

recover plaintext:

$$\boxed{P_i = C_i \oplus k_i}$$

Why?

$$C_i = P_i \oplus k_i$$

$$C_i \oplus k_i = (P_i \oplus k_i) \oplus k_i$$

Encryption Numerical Example

Suppose:

$$P = 10110010$$

and RC4 key-stream byte is:

$$k = 01101001$$

Then:

$$C = P \oplus k$$

$$10110010$$

$$\oplus \quad 01101001$$

$$= \boxed{11011011}$$

Decryption Numerical Example

Given ciphertext:

$$C = 11011011$$

and key-stream byte:

$$k = 01101001$$

Recover:

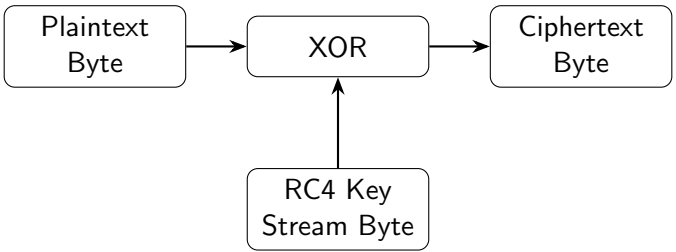
$$P = C \oplus k$$

$$11011011$$

$$\oplus \quad 01101001$$

$$= \boxed{10110010}$$

RC4 Encryption and Decryption Summary



$$C = P \oplus k$$

$$P = C \oplus k$$

RC4 Algorithm Summary

1. Initialize state vector S.
2. Fill temporary vector T using the secret key.
3. Use KSA to permute S.
4. Use PRGA to generate key-stream bytes.
5. XOR key-stream bytes with plaintext bytes.
6. For decryption, XOR ciphertext with the same key stream.

Strengths of RC4

RC4 was popular because:

- It is simple.
- It is fast in software.
- It uses byte-oriented operations.
- It supports variable key size.
- Encryption and decryption are simple XOR operations.

Fast + Simple + Software Friendly

Weaknesses of RC4

RC4 has several weaknesses:

- Early key-stream bytes can be biased.
- Incorrect key handling can make systems vulnerable.
- It was used insecurely in WEP.
- It is not recommended for modern secure communication.
- It has been prohibited in modern TLS usage.

Important

RC4 is studied for understanding stream ciphers, not for recommending new deployment.

RC4 and WEP

RC4 was used in WEP for wireless security.
However, WEP became insecure.
The problem was not only RC4 itself, but also:

- poor key management,
- weak initialization-vector use,
- repeated or related keys,
- protocol-level design weaknesses.

Strong algorithm design must be combined with secure protocol design.

Why RC4 is Deprecated

RC4 is deprecated because practical attacks exploit weaknesses in its output stream. Modern secure protocols avoid RC4.

Better modern choices include:

- AES in secure modes
- ChaCha20

Do not use RC4 in new cryptographic systems.

RC4 vs DES

Feature	**RC4**	**DES**
Cipher type	Stream cipher	Block cipher
Unit	Byte stream	64-bit block
Main operation	XOR with key stream	Feistel rounds
Key size	Variable	56 effective bits
Speed	Very fast in software	Slower than RC4
Modern status	Deprecated	Obsolete

Quick Classroom Questions

1. Who designed RC4?
2. Is RC4 a block cipher or a stream cipher?
3. What is the size of the RC4 state vector?
4. What values does S contain initially?
5. What is the purpose of vector T?
6. What does KSA stand for?
7. What does PRGA stand for?
8. How does RC4 encrypt one byte?
9. Why is XOR used in RC4?
10. Why is RC4 not recommended today?

Practice Numerical 1

Given:

$$P = 11001010$$

and key-stream byte:

$$k = 10110101$$

Find:

$$C = P \oplus k$$

Task

Perform XOR bit by bit and write the ciphertext.

Practice Numerical 1: Solution

$$P = 11001010$$

$$k = 10110101$$

$$C = P \oplus k$$

$$11001010$$

$$\oplus \quad 10110101$$

$$= \boxed{01111111}$$

Therefore:

$$C = 01111111$$

Practice Numerical 2

Given ciphertext:

$$C = 01111111$$

and key-stream byte:

$$k = 10110101$$

Recover:

$$P = C \oplus k$$

Task

Verify that the original plaintext is obtained.

Practice Numerical 2: Solution

$$C = 01111111$$

$$k = 10110101$$

$$P = C \oplus k$$

$$\begin{array}{rr} & 01111111 \\ \oplus & 10110101 \end{array}$$

$$= \boxed{11001010}$$

Therefore:

$$P = 11001010$$

Important Formulas

Initialization

$$S[i] = i$$

$$T[i] = K[i \bmod \text{keylen}]$$

KSA

$$j = (j + S[i] + T[i]) \bmod 256$$

$$\text{Swap}(S[i], S[j])$$

PRGA

$$i = (i + 1) \bmod 256$$

$$j = (j + S[i]) \bmod 256$$

References

1. William Stallings, *Cryptography and Network Security: Principles and Practice*, Pearson.
2. William Stallings, *Computer Security: Principles and Practice*, Pearson.
3. RFC 7465, *Prohibiting RC4 Cipher Suites*.
4. Alfred J. Menezes, Paul C. van Oorschot, Scott A. Vanstone, *Handbook of Applied Cryptography*.

Your Feedback Matters!

Please scan the QR code and share your feedback.



TCS-392 – Introduction to Cryptography
Lecture 16 – RC4 Algorithm

Thank You

Questions?